

Professional Certificate Programme in Agentic AI and Applications



Prompt Engineering and Adaptive Instructions



Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

- How goal-setting works in autonomous agents and its impact on decision-making
- Real-world examples of AI agents solving challenges across industries
- Hierarchical v. reactive planning in AI agents and their impact on decisions
- Lessons from success stories and failures in AI agent implementation
- How AI agents automate workflows and reduce manual tasks
- Multi-agent planning strategies and their coordination
- Reinforcement loops in AI planning and continuous improvement

Topics Covered

- Prompt Engineering Fundamentals
- The Three Core Prompt Types
- Chain-of-Thought Prompting
- Instruction Tuning Basics
- Dynamic Prompt Construction
- Modular Prompt Assembly

Prompt Engineering & Adaptive Instructions

Master the art of communicating with AI systems through strategic prompt design. Transform from beginner to expert in 6 hours of hands-on training.

Welcome to Your Prompt Engineering Journey

Master the essential skill every AI professional needs: prompt engineering



- Apply it whether you're a product manager, ML engineer or data scientist.
- Gain practical, job-ready skills tailored to real-world applications.
- Start from zero assumptions and build up to advanced techniques.
- Learn through hands-on exercises, real-world examples and immediate application.

Learning Roadmap

Foundation building

Master basic prompt types and core principles that drive consistent results



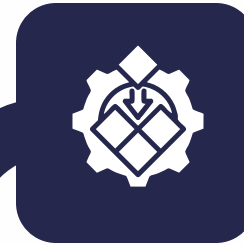
Strategic reasoning

Learn chain-of-thought prompting to guide AI through complex multi-step processes



Adaptive design

Build dynamic prompts that adjust to context and deliver flexible responses



Advanced assembly

Create modular prompt systems using professional frameworks and tools



Practical application

Debug real scenarios and refine prompts through systematic improvement methods

What You'll Achieve Today

Write professional prompts

- ✓ Craft clear, effective prompts that deliver consistent, high-quality outputs across AI models and use cases.

Debug failed interactions

- ✓ Identify why prompts fail and systematically improve them using proven troubleshooting techniques.

Build adaptive systems

- ✓ Design prompt frameworks that adjust to different users, contexts and business needs automatically.

Prompt Engineering Fundamentals

Understanding how AI models process instructions and building your foundation for success

Why Prompt Engineering Matters

Well-crafted prompts produce reliable results, minimising retries and revisions

Consistency



Efficiency

Effective prompts save time and resources by yielding accurate outputs on the first attempt

Structured prompt design supports automation and integration into broader systems

Scalability



What Is a Prompt?

Serves as the interface with an AI model through instructions, context and examples

Sets expectations and guides the AI's response

Initiates every interaction with ChatGPT, Claude or similar models

Directly influences the quality of the response received

Example: Instead of **"Write about marketing,"** say **"Write a 200-word email to enterprise clients explaining how our new analytics feature reduces reporting time by 40%."**

In class Activity: Write a Prompt For Each Task

Example Tasks



Summarise today's news in 3 bullet points.



Translate a sentence into French.



Write a short story about a robot making friends.



Brainstorm 3 healthy breakfast ideas.



Generate Python code to calculate the factorial of a number.



Explain photosynthesis for a 10-year-old.



Draft a professional email to request a meeting.

In Class Activity: Write a Prompt for Each Task

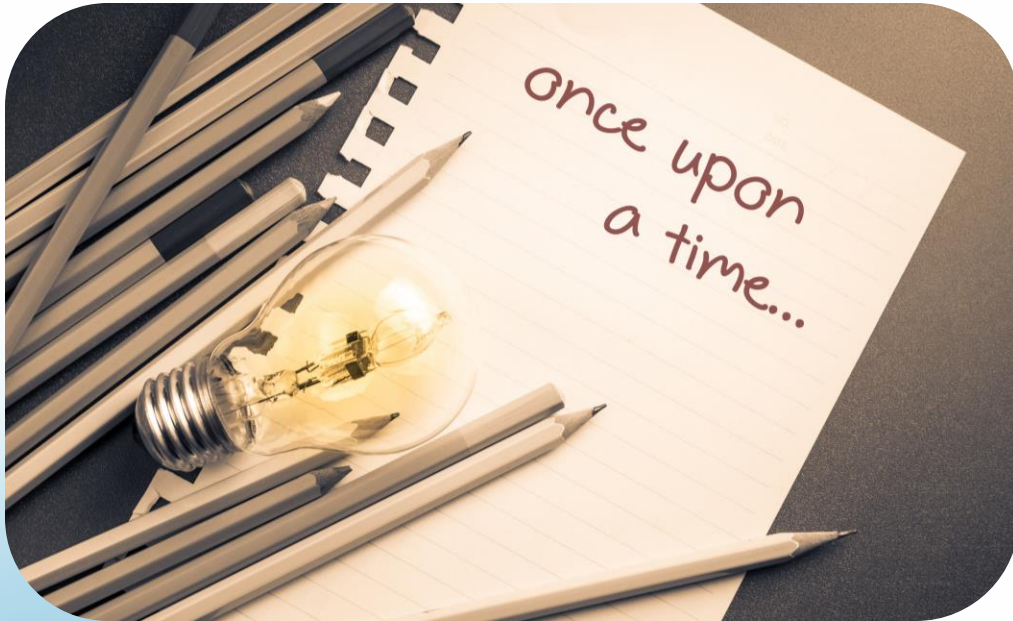
Task 1: Summarise Today's News



- ✓ **Bad prompt:** "Tell me the news."
 - Too vague, no timeframe, scope or structure
- ✓ **Good prompt:** "Summarise today's top 3 global news stories in bullet points, each under 15 words."
 - Specific (3 stories), clear scope (global news), structured output (bullets, word limit)

In Class Activity: Write a Prompt for Each Task

Task 2: Write a Short Story About a Robot Making Friends



- ✓ **Bad prompt:** "Write about robots."
 - No theme, no genre, no length guidance.
- ✓ **Good prompt:** "Write a 150-word children's story about a lonely robot who makes a new friend at school."
 - Provides theme (friendship), audience (children), length (150 words), setting (school)

In Class Activity: Write a Prompt for Each Task

Task 3: Generate Python Code for Factorial



✓ **Bad prompt:** “Do factorial.”

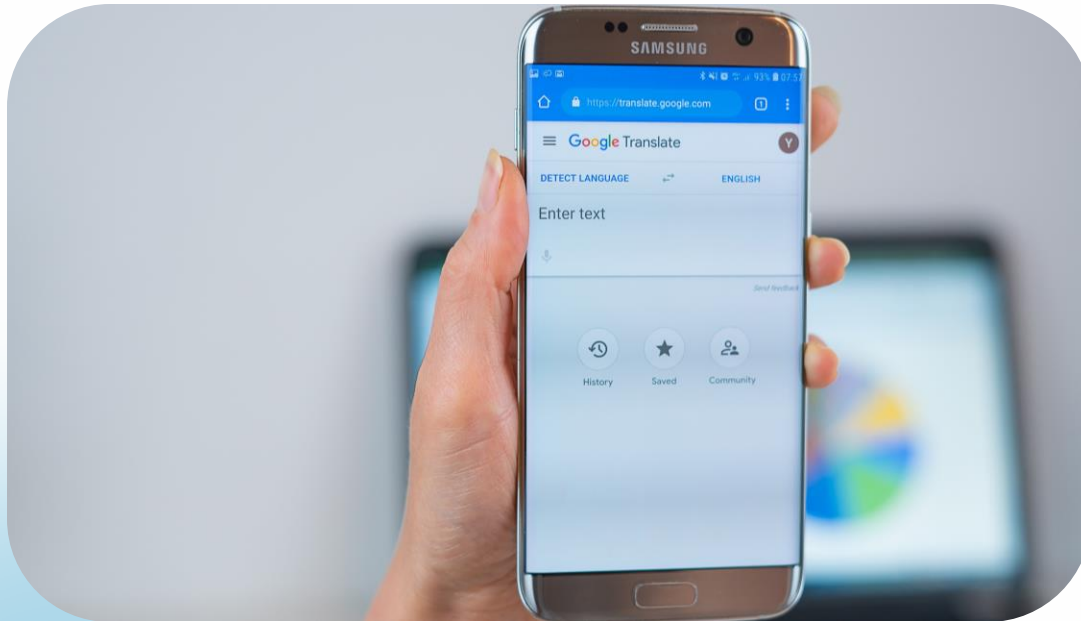
- Too short, no language, no context

✓ **Good prompt:** “Write a Python function called factorial that takes an integer n and returns its factorial using recursion.”

- Programming language, function name, input/output, method (recursion)

In Class Activity: Write a Prompt for Each Task

Task 4: Translate into French



✓ **Bad prompt:** “Translate this.”

- No target language specified, unclear input

✓ **Good prompt:** “Translate the sentence ‘How are you today?’ into French, keeping it formal.”

- Clear sentence, target language, tone (formal)

In Class Activity: Write a Prompt for Each Task

Task 5: Brainstorm Healthy Breakfast Ideas



✓ **Bad prompt:** “Breakfast ideas.”

- Too broad, no context or focus

✓ **Good prompt:** “Suggest 3 healthy breakfast ideas high in protein, suitable for a vegetarian diet.”

- Scope (3 ideas), constraints (healthy, protein, vegetarian)

In Class Activity: Write a Prompt for Each Task

Task 6: Explain Photosynthesis



- ✓ **Bad prompt:** “What is photosynthesis?”
 - Generic, likely to get a textbook definition
- ✓ **Good prompt:** “Explain photosynthesis in simple terms for a 10-year-old using a short analogy.”
 - Defines audience (10-year-old), tone (simple), method (analogy)

In Class Activity: Write a Prompt for Each Task

Task 7: Draft a Professional Email



✓ **Bad prompt:** “Write an email.”

- No purpose, no audience, no style guidance

✓ **Good prompt:** “Draft a professional email requesting a 30-minute meeting with Dr. Smith next week to discuss my research project.”

- Clear recipient, purpose, tone (professional), timeframe

The Three Core Prompt Types

Understanding these fundamental approaches will form the backbone of every prompt you write. Each serves different purposes and offers unique advantages in various scenarios.

Zero-Shot Prompting



Definition:

- Asking the AI to perform a task without providing any examples
- Relying entirely on the model's pre-trained knowledge and clear instructions



When to use:

- Simple, straightforward tasks
- Well-defined problems
- When examples aren't available

Example:

Summarise this product review in one sentence, focusing on the main concern

- No examples provided - the AI uses its training to understand what a summary should look like.

One-Shot Prompting



Definition:

- Providing exactly one example to demonstrate the desired format, style or approach before asking for the actual task



When to use:

- Specific formatting requirements
- Unique style or tone
- Custom output structure

Example: Convert customer feedback to action items

- Feedback: 'The app crashes when I upload large files'
- **Action:** Create ticket for mobile team
- investigate file upload size limits
- **Now convert:** 'Login takes forever on mobile'

Few-Shot Prompting



Definition:

- Providing multiple examples (typically 2-5) to establish a clear pattern before requesting the actual task
- Helping the AI understand nuanced task requirements.



When to use:

- Complex pattern recognition
- Nuanced classification
- Consistent formatting across variations

Example: Classify customer sentiment

- 'Love the new features!' → Positive
- 'App is okay, nothing special' → Neutral
- 'Terrible update, bugs everywhere' → Negative
- 'Pretty good but needs work' → Mixed
- **Classify: 'Great idea but execution is lacking'**

Choosing the Right Prompt Type

1. Start with zero-shot

- ✓ Use clear instructions without examples for simple tasks

2. Add one example

- ✓ Provide a single example if outputs are inconsistent or poorly formatted

3. Scale to few-shot

- ✓ Include 2–4 examples to handle complex patterns or edge cases

Hands-On Exercise: Prompt Type Selection



Your challenge

Classify support tickets into: Technical, Billing, Feature Request or General



Examples

- “My payment failed but I was charged” → Billing
- “Can you add dark mode?” → Feature Request
- “App won't open on iPhone” → Technical
- “How do I change my password?” → General



Your task

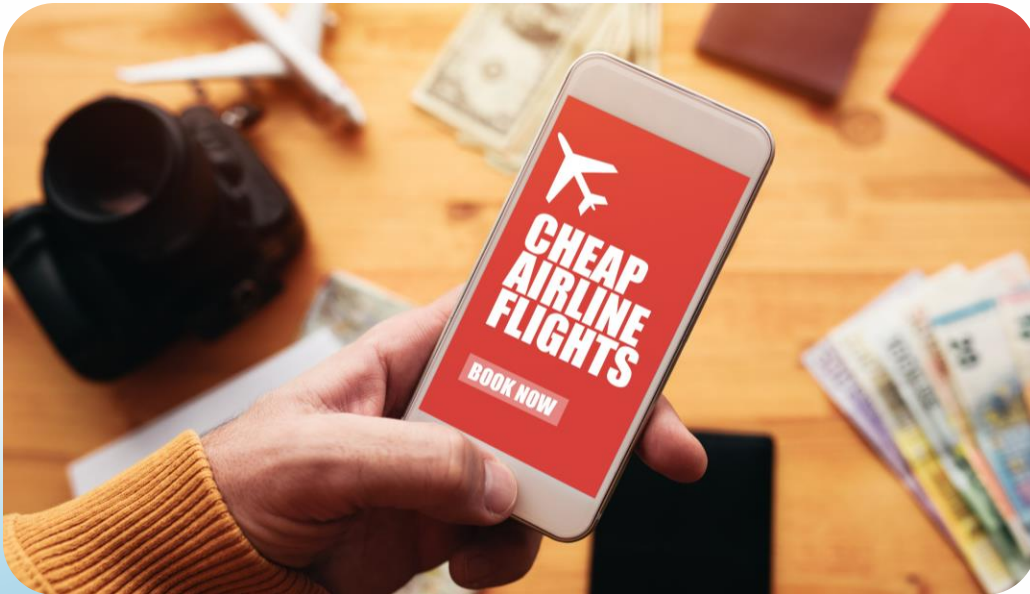
Write three different prompts:

- Zero-shot version
- One-shot version
- Few-shot version

Which approach would work best for this classification task and why?

Exercise Solution: Zero-Shot

Classify This Customer Support Ticket Into One of These Categories



- **Technical:** Issues with app functionality, bugs or performance
- **Billing:** Payment problems, subscription questions, refunds
- **Feature request:** Suggestions for new features or improvements
- **General:** Account questions, how-to inquiries, general support

Ticket: "My payment failed but I was charged"



Strengths: Simple and clear



Weakness: May struggle with edge cases or ambiguous tickets

Exercise Solution: One-Shot

Classify Customer Support Tickets Into Categories

Example: Ticket: "Can you add dark mode?"

Category: Feature request



- **Technical:** App functionality, bugs, performance issues
- **Billing:** Payment problems, subscription questions
- **Feature request:** Suggestions for new features
- **General:** Account questions, how-to inquiries

Ticket: "My payment failed but I was charged"



Strengths: Shows exact format expected



Best for: When you need consistent formatting

Exercise Solution: Few-Shot

Classify Customer Support Tickets

Examples:

- “Can you add dark mode?” → Feature request
- “App won't open on iPhone” → Technical
- “My payment failed but I was charged” → Billing
- “How do I change my password?” → General



- **Technical**– App functionality, bugs or performance issues
- **Billing**– Payment, subscription or refund issues
- **Feature request**– New feature suggestions
- **General**– Account questions or how-to inquiries

Ticket: "The search feature is really slow"



Best choice: Few-shot works best as it guides the AI through varied ticket types and edge cases.

Chain-of-Thought Prompting

Teaching AI to think step-by-step for complex reasoning tasks

What Is Chain-of-Thought Prompting



- Chain-of-thought prompting guides the AI through multi-step reasoning by including intermediate steps.
- The model "thinks out loud" instead of jumping directly to the final answer.

This approach improves accuracy on tasks involving math, logic, analysis or sequential reasoning.

Why Chain-of-Thought Works

Splits complex problems into smaller, sequential steps that build on each other

Breaks down complexity



Reveals reasoning

Shows how the AI reached its conclusion, making errors easier to identify and correct

Boosts performance by 20–50% on reasoning tasks using chain-of-thought prompting

Improves accuracy



Two Chain-of-Thought Approaches

Explicit Chain-of-Thought

You provide examples showing step-by-step reasoning and ask the AI to follow the same pattern.

Q: A store has 23 apples. They sell 8 and receive 15 more. How many total?

A: Starting amount: 23 apples
Sold: $23 - 8 = 15$ apples
Added: $15 + 15 = 30$ apples
Answer: 30 apples

Implicit Chain-of-Thought

You add "Let's think step by step" or "Show your work" to trigger the reasoning process.

Calculate the monthly cost savings if we switch from Plan A (\$299/month) to Plan B (\$199/month) for 50 users.

Let's think step by step:

Real-World Example: Customer Analysis

- ✓ **Business challenge:** Analyse customer feedback to determine if they're likely to churn and what actions to take.

Without Chain-of-Thought

Customer feedback

1. "The app is okay but I'm not using most features."
2. Pretty expensive for what I get.
3. "Will they churn? What should we do?"

Typical response:

"High churn risk. Offer discount."

With Chain-of-Thought

Analyse this customer feedback step by step:

1. Identify sentiment indicators
2. Assess usage patterns
3. Evaluate value perception
4. Determine churn probability
5. Recommend specific actions

Customer says:

"The app is okay but I'm not using most features. Pretty expensive for what I get."

Chain-of-Thought in Action

Scenario

- You are part of a customer experience (CX) transformation team in a telecom company
- Your leadership aims to reduce churn using GenAI insights from call centre transcripts.



Naïve prompt:

“Analyse customer call transcripts and tell me why customers are leaving.”

“What’s wrong with this approach?”

Chain-of-Thought Prompt

**Guide the model on how to think,
not just what to output**



- Identify emotional cues or dissatisfaction signals in transcripts
- Cluster similar complaints into themes
- Quantify how often each theme appears
- Infer the top 3 drivers for churn with evidence
- Suggest actionable interventions for each

**Think step by step like a
CX analyst**



- Detect emotional or dissatisfaction signals in the transcripts
- Group similar causes into categories
- Summarise frequency of each theme
- Identify top 3 churn drivers with supporting evidence
- Recommend actions to address them

Chain-of-Thought Response Example

[Use Chain of Thought Demo Code File](#)

Step-by-step analysis involves:

Sentiment indicators

- ✓ “okay” = neutral/lukewarm sentiment
- ✓ “not using most features” = low engagement
- ✓ “pretty expensive” = price sensitivity

Usage patterns

- ✓ Limited feature adoption suggests onboarding issues
- ✓ Low utilisation indicates poor value realisation

Value perception

- ✓ Customer sees cost but not equivalent benefit
- ✓ Gap between what they pay v. what they use

Chain-of-Thought Response Example

[Use Chain of Thought Demo Code File](#)

Step-by-step analysis involves:

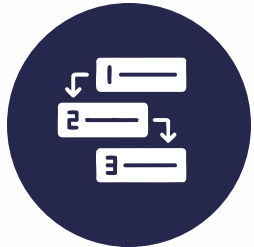
Churn probability

- ✓ HIGH (75–80%)- Classic signs: low engagement + price sensitivity

Recommended actions

- ✓ Schedule feature walkthrough call
- ✓ Offer usage analytics to show hidden value
- ✓ Consider tier downgrade v. discount
- ✓ Focus on 2–3 most relevant features

When to Use Chain-of-Thought



Multi-step calculations

Financial modelling, resource planning, metric analysis or any task involving sequential math operations



Complex comparisons

Vendor selection, feature prioritisation or any task requiring weighing multiple factors against each other



Logical analysis

Root cause analysis, troubleshooting, decision trees or evaluating multiple criteria systematically



Process documentation

Breaking down workflows, creating standard operating procedures or explaining complex methodologies

Instruction Tuning Basics

Fine-tuning AI behaviour through precise instruction design

Understanding Instruction Tuning



- Modifies AI behaviour using guidelines, constraints and examples within prompts
- Works instantly with existing AI systems—no retraining required
- Acts like a job description, defining what to do, how to do it, what to avoid and what success looks like



System instructions

Define role and general behaviour



Task instructions

Specify what to accomplish



Output instructions

Control format and style

Components of Effective Instructions

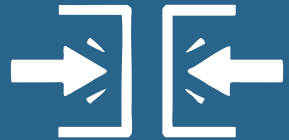


Role definition

Tell the AI what kind of expert it should be: "You are a senior data analyst with 10 years of experience in e-commerce analytics."

Clear objectives

Specify exactly what you want accomplished: "Your goal is to identify the top 3 factors causing cart abandonment."



Constraints & boundaries

Define what NOT to do: "Do not include personally identifiable information. Focus only on aggregate patterns."

Output format

Specify how results should look: "Present findings as: Problem, Evidence, Impact, Recommendation."



Role-Based Instruction Tuning

Different Roles Produce Dramatically Different Outputs

Here's the same task with different role instructions:



As a marketing manager

- You are a marketing manager.
- Analyse this campaign data and focus on ROI, audience segments and optimisation opportunities for future campaigns.



As a data scientist

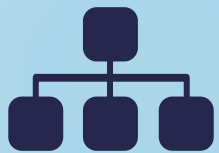
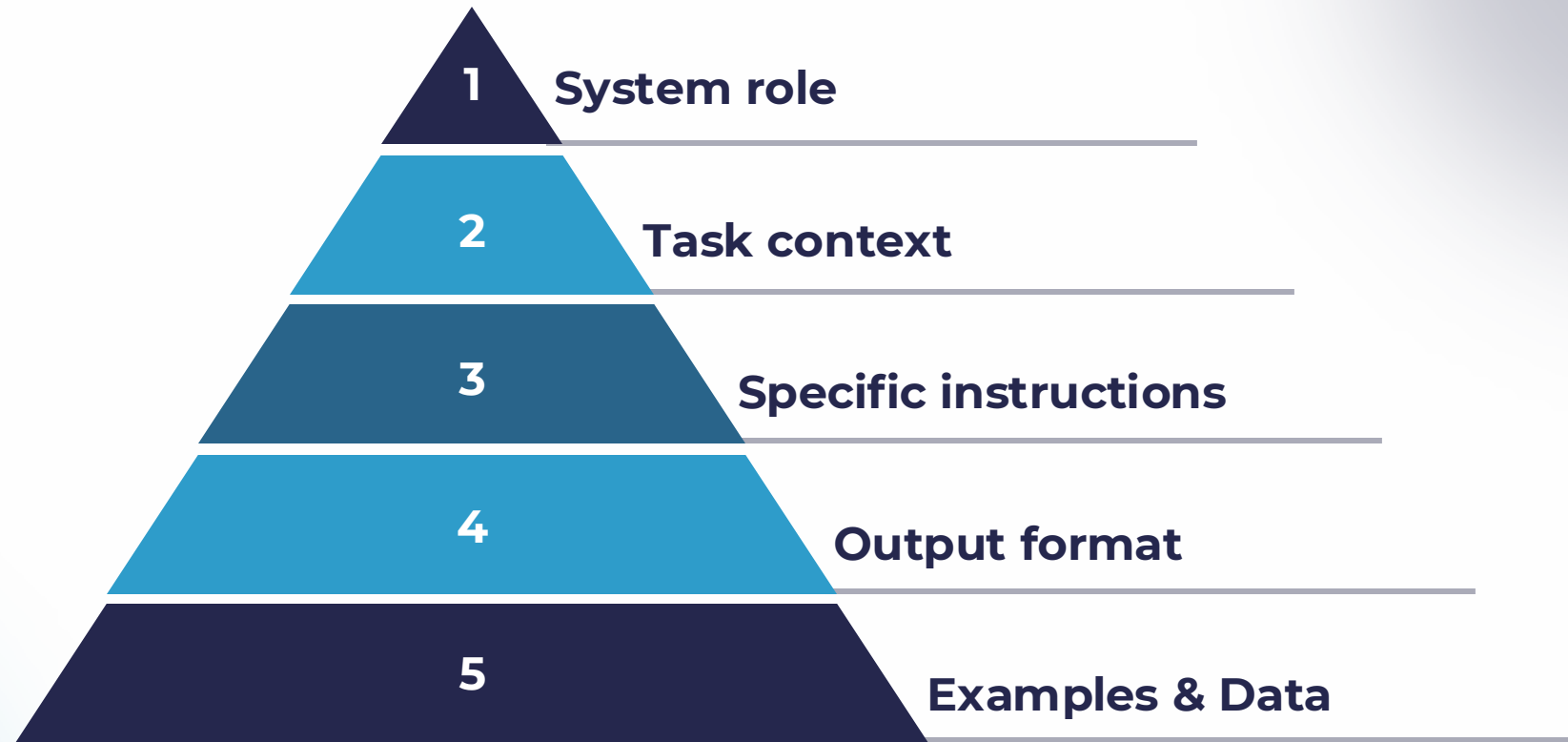
- You are a data scientist.
- Analyse this campaign data and focus on statistical significance, model performance and predictive insights for optimisation.



As a business analyst

- You are a business analyst.
- Analyse this campaign data and focus on business impact, cost efficiency and strategic recommendations for stakeholders.

Instruction Hierarchy



Structure your prompts in this order for maximum effectiveness.



Place system-level instructions first to influence all following processing.

Practical Example: Customer Support Bot



Before: Basic instructions

- Answer customer questions about our product.

After: Tuned instructions

- You are a helpful customer support specialist for TechFlow, a project management software company.
- You are patient, professional and solution-focused.

Practical Example: Customer Support Bot

After: Tuned instructions



Guidelines:

- Always acknowledge the customer's concern first
- Provide specific, actionable solutions
- If you don't know something, say so and offer to escalate
- Keep responses under 150 words
- End with "Is there anything else I can help you with?"



NEVER:

- Make promises about timelines without checking with development
- Provide refunds (escalate to billing team)
- Share information about unreleased features

Output Format Control

After: Tuned instructions

Precise output formatting ensures consistency across your team and systems through proven patterns.



Structured analysis format



Format your response as:

Summary[One sentence overview]

Key Findings

[Finding 1 with evidence]

[Finding 2 with evidence]

[Finding 3 with evidence]

Recommendations

[Action item with owner]

[Action item with owner]

Next Steps[Immediate next action]

Output Format Control

Demonstrates how structured instructions transform LLM outputs

- (Use instruction tuning code file)

Common Instruction Tuning Mistakes

Bad:

"You are helpful"

Good:

"You are a senior product manager with expertise in SaaS metrics and user experience optimization"



Vague role definitions



Missing constraints



Problem:

AI gives overly long responses or includes irrelevant information

Solution:

"Keep responses under 100 words. Focus only on actionable insights."

Issue:

Unclear what "good" output looks like

Fix:

"Success means providing 3 specific, measurable recommendations with implementation timeframes."



No success criteria



Optional Exercise

Scenario: You need an AI assistant to help your team write better product requirements documents (PRDs).

Your task

Create instruction tuning for an AI that will:

- Review draft PRDs for completeness
- Suggest improvements
- Ensure consistency with company standards
- Focus on user impact and technical feasibility

Include: Role, objectives, constraints and output format

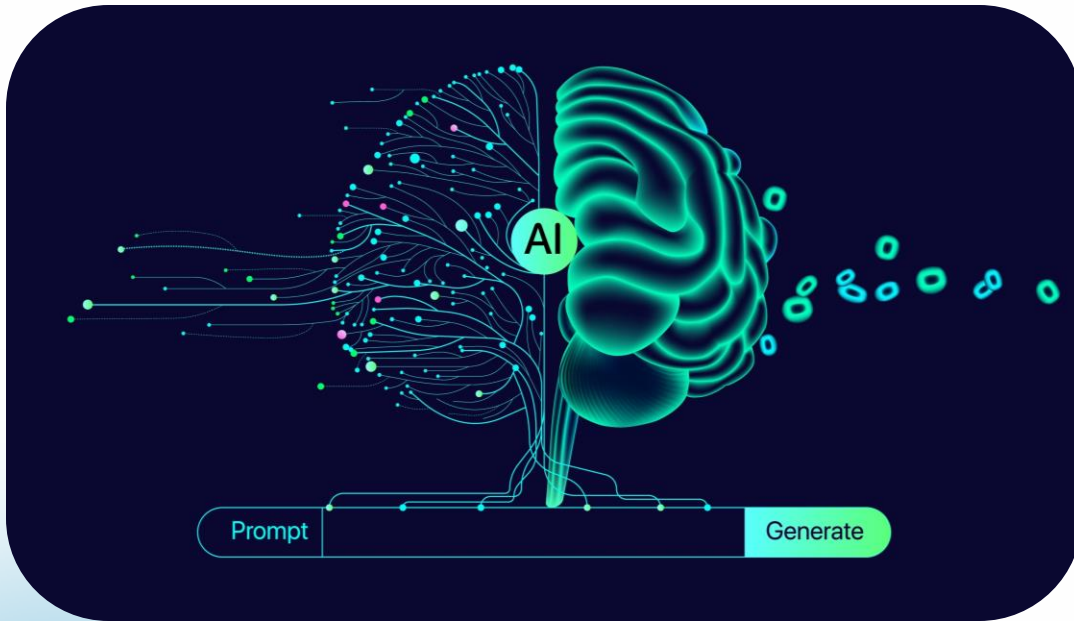


Consider what expertise level would be most helpful and what specific problems your team faces with PRDs.

Dynamic Prompt Construction

Building flexible prompts that adapt to context and user needs

What Are Dynamic Prompts



- ✓ Serve as template-based structures that automatically adjust content, instructions and examples based on input variables, user context or environmental conditions
- ✓ Replace separate scenario-specific prompts by providing one intelligent prompt that adapts
- ✓ Support efficient scaling, maintain consistency, and handle diverse use cases



Template-based



Variable-driven



Context-aware

Dynamic Prompt Patterns

Conditional logic

Instructions that change based on input characteristics:

- ✓ "If technical audience, include implementation details."
- ✓ "If executive audience, focus on business impact."

Variable substitution

Placeholder values that get replaced:

- ✓ "{USER_ROLE} analysing {DATA_TYPE} for {BUSINESS_GOAL}" becomes "Marketing Manager analysing campaign data for ROI optimisation"

Context-aware examples

Examples that match the user's specific situation:

- ✓ SaaS examples for SaaS companies
- ✓ Healthcare examples for medical contexts

Building Your First Dynamic Prompt

Use case: Email response generator that adapts to relationship level and urgency



Variables to consider:

- Recipient relationship (colleague, client, executive)
- Urgency level (low, medium, high)
- Email type (request, update, problem)
- Company tone (formal, casual, friendly)

Dynamic elements:

- Greeting formality
- Level of detail
- Call-to-action strength
- Sign-off style

Dynamic Email Prompt Template

Scenario: You are a Professional Communication Assistant

Generate an email response based on these parameters:

RECIPIENT: {RELATIONSHIP}

URGENCY: {URGENCY_LEVEL}

EMAIL_TYPE: {EMAIL_TYPE}

COMPANY_TONE: {COMPANY_TONE}

INSTRUCTIONS:-

If RELATIONSHIP = "Executive": Use formal tone, be concise, focus on business impact

If RELATIONSHIP = "Colleague": Use collaborative tone, include context, ask for input

If RELATIONSHIP = "Client": Use a professional but warm tone, emphasise service

If URGENCY = "High": Use direct subject line, clear deadlines, specific actions

If URGENCY = "Medium": Use informative subject, suggested timelines, options

If URGENCY = "Low": Use a friendly subject, flexible timelines, open discussion

FORMAT:

Subject: [Clear, appropriate to urgency]

Body: [Greeting] [Context] [Main message] [Action items] [Closing]

EMAIL TO RESPOND TO: {EMAIL_CONTENT}

YOUR RESPONSE:

Advanced Dynamic Patterns

Progressive disclosure

Start simple, add complexity based on user expertise:

- ✓ "If user indicates advanced knowledge, include technical implementation details."

Context stacking

Layer multiple contexts:

- ✓ "Consider {USER_ROLE} in {INDUSTRY} working on {PROJECT_TYPE} with {CONSTRAINTS}."

Feedback loops

Adjust based on user responses:

- ✓ "If user asks for more detail, provide {EXTENDED_ANALYSIS}. If satisfied, proceed to {NEXT_STEPS}."

Implementation Strategies



Start with templates

Begin with static prompts, identify recurring variations and convert repeated elements into variables



Map user contexts

Document user types, use cases and scenarios your prompt must handle



Test edge cases

Verify the logic works with unusual variable combinations and unexpected inputs



Iterate based on usage

Monitor performance and refine the logic using real-world usage patterns

Real-World Example: Support Ticket Routing

A dynamic prompt that automatically categorises and prioritises customer support tickets

Demonstrate how to apply dynamic prompt construction for ticket routing.
(Use dynamic prompt code file)

Optional Exercise

Build a Dynamic Analysis Prompt



Your challenge

Create a dynamic prompt for analysing business metrics that adapts to:

- Different business functions (Sales, Marketing, Product, Operations)
- Various metric types (Revenue, User engagement, Performance, Cost)
- Time periods (Daily, Weekly, Monthly, Quarterly)
- Audience level (Individual contributor, Manager, Executive)

Goal: One prompt that provides relevant analysis regardless of input combination.

Optional Exercise

Build a Dynamic Analysis Prompt



What would change in your analysis approach for a Sales Manager looking at weekly revenue v. a Product Executive reviewing monthly engagement metrics?



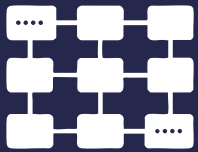
Modular Prompt Assembly

What Is Modular Prompt Assembly?



- Break complex prompts into reusable components that combine, chain and orchestrate into advanced AI workflows
- Replace monolithic prompts by building libraries of prompt modules
- Enable team collaboration, version control and systematic optimisation across use cases

What Is Modular Prompt Assembly?



Reusable components

Mix and match role definitions, instructions and example



Workflow orchestration

Chain multiple prompts to handle complex tasks



Version control

Track changes and optimise individual components

Core Modular Components



Role modules

Standardised expert personas you can apply across different tasks:

"Senior Data Analyst", "UX Researcher", "Technical Writer"

Task modules

Specific instruction sets for common activities:

"Analyse trends", "Generate recommendations", "Create summaries"



Format modules

Output templates that ensure consistency:

"Executive Summary", "Technical Specification", "Action Plan"

Example modules

Industry-specific examples that can be swapped based on context:

"SaaS examples", "Healthcare examples", "Financial examples"



LangChain Framework Overview

(Covered in Previous Module)

LangChain provides tools for chaining prompts, managing context and creating complex AI workflows.

Key LangChain components include:

- **Prompt templates:** Reusable prompt structures with variables
- **Chains:** Sequences of prompts that pass data between steps
- **Memory:** Context retention across interactions
- **Agents:** AI systems that choose which tools to use

```
# Example LangChain Template
template = """You are a {role} with {years} years of
experience in {domain}.Task:
{task_description}Context: {context}Output
format: {output_format}{input}"""
prompt = PromptTemplate( variables=["role",
"years", "domain", "task_description",
"context", "output_format", "input"],
template=template)
```

Building Prompt Libraries

Organisation Structure

```
prompt-library/  
├── roles/  
│   ├── data-analyst.md  
│   ├── product-manager.md  
│   └── technical-writer.md  
├── tasks/  
│   ├── analyze-trends.md  
│   ├── create-summary.md  
│   └── generate-recommendations.md  
├── formats/  
│   ├── executive-brief.md  
│   ├── technical-spec.md  
│   └── action-plan.md  
└── examples/  
    ├── saas/  
    ├── healthcare/  
    └── finance/
```

Best practices

- Version control all components
- Document performance metrics
- Test modules in isolation
- Create combination guides
- Gather user feedback

Exercise: Design Your Prompt Library

Your challenge: Design a modular prompt system for your most common AI use case.



Consider:

- What roles/expertise do you need regularly?
- What tasks do you repeat frequently?
- What output formats do you use?
- How would you combine modules?

Common Prompt Failures

Inconsistent outputs

Symptoms: Same prompt gives different results each time

Causes: Vague instructions, missing examples, unclear success criteria

Symptoms: AI stops mid-task or skips important elements

Causes: Too many instructions, unclear priorities, context overload

Incomplete responses

Off-target results

Symptoms: Output doesn't match intended purpose or audience

Causes: Wrong role definition, missing context, poor examples

Symptoms: Output ignores specified structure or style

Causes: Conflicting instructions, unclear format requirements

Format violations

Q & A

Thank you